

Kotlin第一章第三节教学讲义（接口与特色类语法）

各位同学，上节课我们掌握了Kotlin类的基础、继承等核心知识，这节课咱们聚焦“接口”和Kotlin特有的类（数据类、密封类等），这些是提升代码灵活性和可读性的关键。尤其是接口，在Android开发中实现“多态”和“解耦”全靠它；而扩展函数、数据类等特色语法，能让你少写很多冗余代码。全程用“比喻+代码”双模式讲解，保证零基础也能吃透！

一、接口：统一“行为标准”的协议

生活中“USB接口”是一个标准——不管是U盘、鼠标还是键盘，只要符合这个标准就能插在电脑上用。Kotlin中的接口（用 `interface` 定义）也是如此，它规定了“类应该有哪些方法”，但不关心这些方法具体怎么实现，实现接口的类必须“遵守协议”，实现所有规定的方法。

核心特点：接口不能实例化（不能new），一个类可以实现多个接口（解决Java单继承的限制）。

1. 接口的基本定义与实现

语法： `interface 接口名 { fun 方法名(参数: 类型) }`，类用 `:` 实现接口（多个接口用逗号分隔）。

代码示例：定义“充电”接口，手机和电脑都实现它

Code block

```
1  // 定义接口：充电协议（规定必须有充电方法）
2  interface Chargeable {
3      // 接口中的方法：只有声明，没有实现
4      fun charge()
5      // 接口中的默认方法（Kotlin特有，加default关键字，实现类可重写）
6      fun showChargeInfo() {
7          println("正在充电中...")
8      }
9  }
10
11 // 手机类：实现充电接口
12 class Phone : Chargeable {
13     // 必须实现接口的抽象方法charge()
14     override fun charge() {
15         println("手机用Type-C接口充电，功率20W")
16     }
17 }
```

```

18 // 可选：重写接口的默认方法
19 override fun showChargeInfo() {
20     super.showChargeInfo() // 调用接口的默认实现
21     println("手机电量已充至50%")
22 }
23 }
24
25 // 电脑类：实现充电接口
26 class Computer : Chargeable {
27     // 实现充电方法（电脑的实现和手机不同）
28     override fun charge() {
29         println("电脑用方口充电器充电，功率100W")
30     }
31     // 不重写showChargeInfo()，直接用接口的默认实现
32 }
33
34 fun main() {
35     val myPhone = Phone()
36     myPhone.charge() // 调用手机的充电实现
37     myPhone.showChargeInfo() // 调用重写后的信息方法
38
39     val myComputer = Computer()
40     myComputer.charge() // 调用电脑的充电实现
41     myComputer.showChargeInfo() // 调用接口的默认实现
42
43     // 输出：
44     // 手机用Type-C接口充电，功率20W
45     // 正在充电中...
46     // 手机电量已充至50%
47     // 电脑用方口充电器充电，功率100W
48     // 正在充电中...
49 }

```

2. 接口属性：规定“必须有”的属性

接口不仅能定义方法，还能定义属性，但接口的属性**不能直接赋值**（因为接口没有“存储能力”），必须由实现类提供具体值或实现getter。

代码示例：带属性的“支付接口”

Code block

```

1 // 支付接口：规定必须有支付方式和支付状态属性
2 interface Payable {
3     // 接口属性1：抽象属性（必须由实现类赋值）
4     val payType: String
5

```

```

6      // 接口属性2: 带默认getter的属性 (实现类可重写)
7      val isPaySuccess: Boolean
8          get() = false // 默认支付未成功
9  }
10
11 // 微信支付: 实现支付接口
12 class WeChatPay : Payable {
13     // 实现接口的抽象属性payType
14     override val payType: String = "微信支付"
15
16     // 重写接口的isPaySuccess属性 (根据实际支付结果返回)
17     override val isPaySuccess: Boolean
18         get() = true // 假设支付成功
19 }
20
21 // 支付宝支付: 实现支付接口
22 class Alipay : Payable {
23     override val payType: String = "支付宝支付"
24     // 不重写isPaySuccess, 默认返回false
25 }
26
27 fun main() {
28     val weChat = WeChatPay()
29     println("支付方式: ${weChat.payType}, 支付状态: ${if (weChat.isPaySuccess) "成功" else "失败"}")
30
31     val alipay = Alipay()
32     println("支付方式: ${alipay.payType}, 支付状态: ${if (alipay.isPaySuccess) "成功" else "失败"}")
33
34     // 输出:
35     // 支付方式: 微信支付, 支付状态: 成功
36     // 支付方式: 支付宝支付, 支付状态: 失败
37 }

```

二、扩展函数：给现有类“加功能”不修改源码

假设你买了一部手机（现有类，比如String），想给它加一个“无线充电”功能（新方法），但又不能拆开手机改硬件（不修改源码）——Kotlin的扩展函数就能实现这个需求。它可以在不继承、不修改原类的前提下，给类新增方法。

语法： `fun 类名.新方法名(参数: 类型): 返回值 { 实现逻辑 }`

1. 给系统类加扩展函数（常用场景）

比如给String类加“判断是否为手机号”的方法，给Int类加“转成金额格式”的方法：

Code block

```
1 // 1. 给String类扩展：判断是否为手机号（简单校验）
2 fun String.isPhoneNumber(): Boolean {
3     // 正则表达式：11位数字，以1开头
4     val regex = "^1[3-9]\\d{9}$".toRegex()
5     return this.matches(regex) // this代表调用该方法的String对象
6 }
7
8 // 2. 给Int类扩展：转成金额格式（比如1000→1,000元）
9 fun Int.toMoney(): String {
10     return "%,d元".format(this) // 千分位分隔符
11 }
12
13 fun main() {
14     val phone1 = "13812345678"
15     val phone2 = "123456"
16     println("$phone1 是否为手机号: ${phone1.isPhoneNumber()}") // true
17     println("$phone2 是否为手机号: ${phone2.isPhoneNumber()}") // false
18
19     val money1 = 5000
20     val money2 = 123456
21     println("金额1: ${money1.toMoney()}") // 5,000元
22     println("金额2: ${money2.toMoney()}") // 123,456元
23 }
```

2. 给自定义类加扩展函数

给上节课的Car类加“鸣笛”功能，不用修改Car类的源码：

Code block

```
1 // 自定义类：汽车（无需修改）
2 class Car(val brand: String) {
3     fun run() = println("$brand 正在跑")
4 }
5
6 // 给Car类扩展鸣笛方法
7 fun Car.horn() = println("$brand 鸣笛：滴滴滴！")
8
9 fun main() {
10     val car = Car("特斯拉")
11     car.run() // 原类方法
12     car.horn() // 扩展方法
13     // 输出：
14     // 特斯拉 正在跑
15     // 特斯拉 鸣笛：滴滴滴！
```

三、数据类：专门存数据的“轻量类”

开发中经常需要“只存数据，不做复杂逻辑”的类（比如用户信息、商品数据），如果用普通类，需要写getter/setter、toString()（打印数据）、equals()（判断相等）等一堆代码。Kotlin的**数据类**（用 `data class` 定义）会自动生成这些方法，直接用就行！

核心要求：主构造器至少有一个参数，且参数必须是val或var（要存数据）。

1. 数据类的基本使用

Code block

```

1  // 数据类：用户信息（自动生成toString、equals、hashCode等方法）
2  data class User(
3      val id: Int,
4      val name: String,
5      var age: Int,
6      val phone: String? = null // 可空属性，默认值null
7  )
8
9  fun main() {
10     // 1. 创建数据类对象
11     val user1 = User(1, "小明", 18, "13812345678")
12     val user2 = User(1, "小明", 18) // 省略默认参数
13     val user3 = User(2, "小红", 20)
14
15     // 2. 自动生成toString(): 直接打印数据，不用手动写
16     println(user1) // 输出: User(id=1, name=小明, age=18, phone=13812345678)
17
18     // 3. 自动生成equals(): 判断内容是否相等（不是地址）
19     println("user1和user2是否相等: ${user1 == user2}") // true (id、name、age都相
    同)
20     println("user1和user3是否相等: ${user1 == user3}") // false
21
22     // 4. 复制对象（数据类特有copy()方法，可修改部分属性）
23     val user4 = user1.copy(age = 19, phone = "13987654321")
24     println("复制后的用户: $user4") // User(id=1, name=小明, age=19,
    phone=13987654321)
25
26     // 5. 解构赋值（数据类特有，把属性拆分成变量）
27     val (id, name, age, phone) = user1
28     println("解构结果: id=$id, name=$name, age=$age, phone=$phone")
29 }

```

2. 数据类在Android中的应用

在Android开发中，网络请求返回的“商品数据”“列表数据”都用数据类接收，比如：

Code block

```
1  // 数据类：商品信息（对应接口返回的JSON格式）
2  data class Goods(
3      val goodsId: String,
4      val goodsName: String,
5      val price: Double,
6      val imageUrl: String,
7      val salesCount: Int // 销量
8  )
9
10 // 用列表存商品数据
11 val goodsList = listOf(
12     Goods("1001", "小米手机", 3999.0, "url1", 10000),
13     Goods("1002", "华为平板", 2999.0, "url2", 5000)
14 )
15
16 // 遍历商品列表
17 fun printGoods() {
18     for (goods in goodsList) {
19         println("商品: ${goods.goodsName}, 价格: ${goods.price}, 销量:
20             ${goods.salesCount}")
21     }
22 }
```

四、密封类：限制“子类范围”的安全类

生活中“交通灯”只有红、黄、绿三种状态，不会出现第四种——Kotlin的密封类（用 `sealed class` 定义）就像交通灯，它的子类必须定义在同一个文件中，这样用when判断时，就能“穷尽所有可能”，避免遗漏情况，代码更安全。

核心特点：密封类本身是抽象的（不能实例化），子类可以是数据类、普通类等。

1. 密封类的基本使用

Code block

```
1  // 密封类：交通灯（限制子类只能是Red、Yellow、Green）
2  sealed class TrafficLight {
3      // 子类1：红灯
4      data class Red(val duration: Int) : TrafficLight() // 红灯持续时间
```

```

5      // 子类2: 黄灯
6      object Yellow : TrafficLight() // 无属性, 用object (单例)
7      // 子类3: 绿灯
8      data class Green(val duration: Int) : TrafficLight()
9  }
10
11 // 处理交通灯状态 (when会自动提示所有子类, 不会遗漏)
12 fun handleTrafficLight(light: TrafficLight) {
13     when (light) {
14         is TrafficLight.Red -> println("红灯, 禁止通行, 持续${light.duration}秒")
15         TrafficLight.Yellow -> println("黄灯, 准备停车")
16         is TrafficLight.Green -> println("绿灯, 可以通行, 持续${light.duration}
秒")
17         // 不需要else, 因为when已经穷尽所有子类
18     }
19 }
20
21 fun main() {
22     val redLight = TrafficLight.Red(60)
23     val yellowLight = TrafficLight.Yellow
24     val greenLight = TrafficLight.Green(40)
25
26     handleTrafficLight(redLight)
27     handleTrafficLight(yellowLight)
28     handleTrafficLight(greenLight)
29
30     // 输出:
31     // 红灯, 禁止通行, 持续60秒
32     // 黄灯, 准备停车
33     // 绿灯, 可以通行, 持续40秒
34 }

```

五、抽象类：“半张图纸”的抽象设计

上节课学的普通类是“完整的图纸”（能直接造对象），而抽象类（用 `abstract` 定义）是“半张图纸”——它包含“未完成的方法/属性”（抽象方法/属性），必须由子类完成这些“未完成部分”才能使用，不能直接实例化。

与接口的区别：接口是“行为标准”，抽象类是“部分实现的类”，一个类只能继承一个抽象类，但可以实现多个接口。

1. 抽象类的定义与继承

Code block

```
1  // 抽象类：动物（半张图纸，有未完成的方法）
2  abstract class Animal(
3      val name: String, // 普通属性（有值）
4      abstract val legCount: Int // 抽象属性（无值，子类必须实现）
5  ) {
6      // 普通方法（有实现）
7      fun eat() {
8          println("$name 在吃东西")
9      }
10
11     // 抽象方法（无实现，子类必须实现）
12     abstract fun move()
13 }
14
15 // 子类1：狗（完成抽象类的未实现部分）
16 class Dog(name: String) : Animal(name, legCount = 4) {
17     override fun move() {
18         println("$name 用4条腿跑")
19     }
20 }
21
22 // 子类2：鸟（完成抽象类的未实现部分）
23 class Bird(name: String) : Animal(name, legCount = 2) {
24     override fun move() {
25         println("$name 用2条腿跳，还能飞")
26     }
27 }
28
29 fun main() {
30     val dog = Dog("旺财")
31     dog.eat() // 调用抽象类的普通方法
32     dog.move() // 调用子类实现的抽象方法
33     println("狗的腿数: ${dog.legCount}")
34
35     val bird = Bird("小鸟")
36     bird.eat()
37     bird.move()
38     println("鸟的腿数: ${bird.legCount}")
39
40     // 输出：
41     // 旺财 在吃东西
42     // 旺财 用4条腿跑
43     // 狗的腿数：4
44     // 小鸟 在吃东西
45     // 小鸟 用2条腿跳，还能飞
46     // 鸟的腿数：2
47 }
```


六、Kotlin特色语法：让代码更简洁

Kotlin有很多“语法糖”，能大幅简化代码，下面是开发中最常用的3个：

1. 空安全相关：?. 和 ?:（避免空指针）

Kotlin的空安全是核心优势，`?.` 表示“对象不为空才调用方法”，`?:` 表示“对象为空就用默认值”。

Code block

```
1 fun main() {
2     val name: String? = null // 可空字符串（可能为null）
3     val age: Int? = 18
4
5     // 1. ?. : name不为空才调用length(), 为空则返回null
6     val nameLength = name?.length
7     println("名字长度: $nameLength") // 输出: 名字长度: null
8
9     // 2. ?: : age不为空就用age的值, 为空就用默认值0
10    val safeAge = age ?: 0
11    println("安全年龄: $safeAge") // 输出: 安全年龄: 18
12
13    // 3. 链式调用: 避免多层空判断
14    val user: User? = User(1, "小明", 18)
15    val userPhone = user?.phone ?: "未绑定手机号"
16    println("用户手机号: $userPhone") // 输出: 用户手机号: 未绑定手机号
17 }
```

2. 区间与集合：in 和 !in

判断“值是否在区间内”或“元素是否在集合中”，比Java简洁很多。

Code block

```
1 fun main() {
2     // 1. 数字区间
3     val score = 85
4     if (score in 60..100) { // 60到100之间（包含边界）
5         println("成绩及格")
6     }
7
8     // 2. 字符区间
9     val c = 'B'
10    if (c in 'A'..'Z') {
```

```

11         println("是大写字母")
12     }
13
14     // 3. 集合判断
15     val fruits = listOf("苹果", "香蕉", "橙子")
16     if ("苹果" in fruits) {
17         println("苹果在水果列表中")
18     }
19     if ("西瓜" !in fruits) {
20         println("西瓜不在水果列表中")
21     }
22 }

```

3. 函数简化：单表达式函数

如果函数体只有一行代码，可以省略大括号和return，直接用 `=` 连接。

Code block

```

1  // 普通函数
2  fun add(a: Int, b: Int): Int {
3      return a + b
4  }
5
6  // 单表达式函数（简化后）
7  fun addSimple(a: Int, b: Int): Int = a + b
8
9  // 甚至可以省略返回值类型（Kotlin自动推断）
10 fun multiply(a: Int, b: Int) = a * b
11
12 fun main() {
13     println(add(1, 2)) // 3
14     println(addSimple(3, 4)) // 7
15     println(multiply(2, 5)) // 10
16 }

```

总结与课后作业

1. 核心知识点回顾

- **接口**：用interface定义，规定行为标准，类可实现多个，支持默认方法和属性。
- **扩展函数**：fun 类名.方法名()，给现有类加功能不修改源码。
- **数据类**：data class，自动生成toString、equals等方法，专门存数据。

- **密封类**：sealed class，子类范围固定，when判断更安全。
- **抽象类**：abstract class，有抽象方法/属性，子类必须实现，只能单继承。
- **特色语法**：?.（空安全调用）、?:（默认值）、in（区间判断）、单表达式函数。

2. 课后作业：实现“电商商品管理”功能

需求：

1. 定义 `Buyable` 接口：包含抽象属性 `price: Double`，方法 `getDiscountPrice(): Double`（计算折扣价）。
2. 定义数据类 `Goods` 实现 `Buyable` 接口，属性包括 `goodsId (String)`、`name (String)`、`price (Double)`、`discount (Double`，折扣率，比如0.8代表8折)，实现 `getDiscountPrice()` 方法（返回 `price*discount`）。
3. 给 `Goods` 类扩展函数 `showInfo()`，打印“商品ID: XXX，名称: XXX，原价: XXX，折扣价: XXX”。
4. 定义密封类 `OrderStatus`，子类包括 `Pending`（待支付）、`Paid`（已支付，带支付时间 `String`）、`Cancelled`（已取消，带取消原因 `String`）。
5. 创建3个 `Goods` 对象，存到列表中，遍历列表调用 `showInfo()`；再创建一个 `OrderStatus` 对象，用 `when` 处理不同状态并打印信息。

提示：折扣价保留两位小数可用 `String.format("%.2f", price)`。

（注：文档部分内容可能由 AI 生成）